Backend Technical Assessment — FastAPI

Task: Build a News Content Management API

Time Limit - 2days

Presentation

After completing the task, you will have **5 minutes** to explain your implementation, decisions, and any unfinished parts.

Scenario

Build a small backend system for managing posts for a social media news page.

The API should allow an administrator to:

- Create news posts
- View all posts
- View a specific post
- Update posts
- Delete posts
- Track post views
- View basic dashboard statistics

No frontend is required.

---

1. News Post Data Model

Each news post should contain the following fields:

```
{
  "id": 1,
  "title": "Pokhara hosts technology event",
  "content": "Full news description",
  "category": "Technology",
  "status": "draft",
  "views": 0,
  "created_at": "2026-07-14T12:00:00"
}
```

Field Requirements

- `id`: Automatically generated
- `title`: Required
- `content`: Required
- `category`: Required
- `status`: Required, with a default value of `draft`
- `views`: Automatically starts at `0`
- `created_at`: Automatically set when the post is created

The `status` field must accept only:

draft
published
archived

---

2. Required API Endpoints

Create a News Post

POST /posts

The endpoint should:

- Validate the request using Pydantic
- Automatically generate the post ID
- Automatically set the creation date and time
- Set the default view count to zero
- Return the created post
- Use an appropriate HTTP status code

Example request:

```
{
  "title": "Pokhara hosts technology event",
  "content": "Full news description",
  "category": "Technology",
  "status": "draft"
}
```

Get All News Posts

GET /posts

The endpoint should return all available posts.

It must also support the following optional filters:

GET /posts?category=Technology
GET /posts?status=published
GET /posts?search=pokhara

The `search` parameter should search inside both:

- Post title
- Post content

The filters should also work together where possible.

Example:

GET /posts?category=Technology&status=published&search=pokhara

---

Get a Single News Post

GET /posts/{post_id}

The endpoint should:

- Return the requested post
- Return a `404 Not Found` response when the post does not exist

Update a News Post

PUT /posts/{post_id}

The endpoint should allow the following fields to be updated:

- Title
- Content
- Category
- Status

The endpoint should:

- Validate the updated information
- Return the updated post
- Return `404 Not Found` when the post does not exist
- Reject invalid status values

Delete a News Post

DELETE /posts/{post_id}

The endpoint should:

- Delete the requested post
- Return a meaningful success response
- Return `404 Not Found` when the post does not exist

Example response:

```
{
  "message": "Post deleted successfully"
}
```

Increase Post Views

POST /posts/{post_id}/view

Each request to this endpoint should:

- Increase the post's `views` value by one
- Save the updated value in the database
- Return the updated post or updated view count
- Return `404 Not Found` when the post does not exist

Example response:


```
{
  "post_id": 1,
  "views": 11
}
```

Dashboard Statistics

GET /dashboard/stats

The endpoint should return:

- Total number of posts
- Total published posts
- Total draft posts
- Total archived posts
- Total views across all posts
- Most viewed post

Example response:


```
{
  "total_posts": 12,
  "published_posts": 8,
  "draft_posts": 3,
  "archived_posts": 1,
  "total_views": 15420,
```

```
  "most_viewed_post": {
    "id": 4,
    "title": "Breaking News",
    "views": 5400
  }
}
```

When there are no posts, the API should handle the response properly without crashing.

---

3. Technical Requirements

Use the following technologies:

- FastAPI
- Python
- Pydantic
- SQLite
- SQLAlchemy
- Uvicorn

The application should also include:

- Proper HTTP status codes
- Basic exception handling
- Clear error messages
- Request validation
- Database persistence
- Automatically generated Swagger documentation at:

/docs

---

4. Suggested Project Structure

Use the following structure where possible:

```
news_api/
├── main.py
├── database.py
├── models.py
```

```
├── schemas.py
├── requirements.txt
└── README.md
```

A simpler structure is acceptable if the application is functional, understandable, and cleanly written.

---

5. Deliverables

Submit the following:

- Complete source code
- `requirements.txt`
- `README.md`
- SQLite database file, if generated
- Screenshots or exported results showing the API working in Swagger or Postman

The `README.md` should include:

- Project overview
- Installation steps
- How to create and activate the virtual environment
- How to install dependencies
- How to run the server
- Available endpoints
- Example requests
- Any unfinished features
- Any assumptions made

Example command:

uvicorn main:app --reload

---

6. Bonus Tasks

Attempt these only after completing the required features.

- Add pagination using `skip` and `limit`
- Sort posts by views or creation date

- Prevent duplicate post titles
- Add API-key authentication for create, update, and delete endpoints
- Add one or two automated tests using `pytest`
- Add a partial update endpoint using `PATCH`
- Add an `updated_at` field
- Use environment variables for configuration

Incomplete bonus features will not reduce the score.

---

7. Evaluation Criteria

Area	Marks
API functionality	30
FastAPI and Pydantic usage	15
Database implementation	15
Validation and error handling	15
Code structure and readability	10
Filtering and statistics logic	10
Explanation of decisions	5
Total	**100**